

Second-order Markov Chain of Computer Logs

Bai, Yilei, bai.yil@northeastern.edu

Feit, Maxwell, feit.m@northeastern.edu

Sachleben, Tariq, sachleben.t@northeastern.edu

Abstract

System logs are a profound corpus of system behavioral information. The realm of identifying and classifying these logs is robust, with corpus datasets such as Loghub [Zhu 2023] collating insights about multiple log spaces with their underlying datasets. We extend present core approaches by interpreting log analysis as a prediction problem. By generating an ordinary second-order Markov Chain leveraging a notion of “log type” for our states, we are able to analyze the transition behavior of the log space. We performed all our analysis on the Thunderbird2 dataset provided by USENIX CFDR [CFDR/HPC4/Tbird2] in modest time using off-the-shelf consumer hardware. While predictivity of this system is imperfect, we believe it represents a building block on which more complete approaches can be built.

Introduction

Logs present a profound corpus of system behavioral data, and they are ubiquitous in modern computing systems at all levels as a way to inform about the inner workings of a system, whether the system is low-level hardware or higher-level software. In large software it becomes prohibitively difficult to prove that systems behave correctly at runtime, simply due to the exponentially growing state space involved. Errors are bound to occur. Logging is deployed to assist in design and debugging of systems. Logs can either directly or indirectly signal when and where an error has occurred while a system is running, and thus they are naturally also useful in identifying what caused those errors. Organizations that maintain critical software and hardware infrastructure dedicate enormous resources towards keeping and analyzing logs at scale to promptly catch errors when they occur, as well as to trace and address their root causes, facilitating the prevention of such incidents in the future and ensuring smooth operation of those critical systems.

Much work has been done with the intention of accelerating and scaling up log detection and analysis for use in various domains in both research and industry (e.g. business, cybersecurity). There has been extensive work in investigating methods for efficiently detecting anomalies as well as indexing and analyzing log data. For example, Zhang et al. performed research analyzing log data primarily using Regular Expressions [Zhang 2025]. There is also the previously mentioned work done on Loghub, as well as work by Liu et al. in performing probabilistic classification of anomalous logs [Liu 2020]. However, such work focuses on detecting and responding to errors after the fact, where the consequences of those errors may have already occurred. While logs are incredibly helpful for error detection and post-hoc tracing, we believe there is a lack of research in applying them towards predicting errors before they even occur. If implemented well at scale, error prediction could further expedite the flagging and prevention of errors, potentially reducing resources spent towards not only catching errors, but also cleaning up whatever problems they cause downstream. In this paper, we set out to build a proof-of-concept method for using log data to predict future errors.

In order to use logs towards error prediction, we operate under some assumptions. We take that while computer software is fully deterministic in execution, the inputs of a given software system are fundamentally probabilistic. While the system state at any given time can be computed using the software and a set of time-varying inputs, without knowledge of those inputs, the state itself becomes a probabilistic measure parameterized by the input distribution. In applications, software engineers commonly add logging tools to their software as a way to track both the system’s current status as well as anomalous events within their code bases. Since logging statements are themselves code, we can now take these logs as a measure of the code state, if a coarse one.

Under the assumption that log data can serve as a reliable representation of the underlying state of a computer or piece of software, and given the temporal and sequential nature of those logs, we model system logs as a Markov chain, where the logs themselves are, in effect, the states. Transitions from one log line to another line then parallel transitions from one state to the next. Given the complexity of a computing system’s underlying functions and the repetitive nature of logs, there is a strong likelihood that states from multiple time steps prior would still have an effect on a system’s current state. Thus, a higher-order Markov chain was deemed necessary to try and capture this

reality. For our proof-of-concept model, we use a second-order Markov chain, where the current log, or state of the system, is dependent on both the previous log or state, as well as the log or state prior to that one. To train, test, and validate our model, we used a large set of system log data collected from a supercomputing system called Thunderbird from 2004 through 2006. This dataset is publicly available and has been commonly used in log parsing and abnormality classification research.

Our work demonstrates that there is merit in the predictive approach towards error detection and flagging. The remainder of this paper will elaborate on our methodology, the justification for this approach, the results from testing, and finally, some reflections on our work and future avenues to potentially explore.

Approach

At the highest level, our approach consists of four steps:

1. Feature Extraction: Using log embeddings based on the [Github/Loghub] corpus, [Zhu 2023] we are able to convert raw log data into a more compact “state representation”
2. Training: Transition events are counted and normalized, and the required datasets for interpretation are emitted, discussed below.
3. Interpretation: given a trained model and a set of state vectors, we can either compute the likelihood of a given transition sequence, or predict the likelihood of different emissions given a one- or two- state prior.
4. Evaluation: We validate that our trained model classifies normal and anomalous logs, and test our prediction code against our testing dataset.

We take and will reference below the following variables:

- **alpha**: normalization variable for training low-occurrence logs.
- **count**: total number of logs in a given training set.
- **DIM**: dimensionality of the state space, equal to the number of known log types + 1.
- $\{s\}$: A complete sequence of log events, typically our training or testing subsets.
- s_0, s_1, s_2 : ordinal log events from a sequence, corresponding to the first, second, and third events.

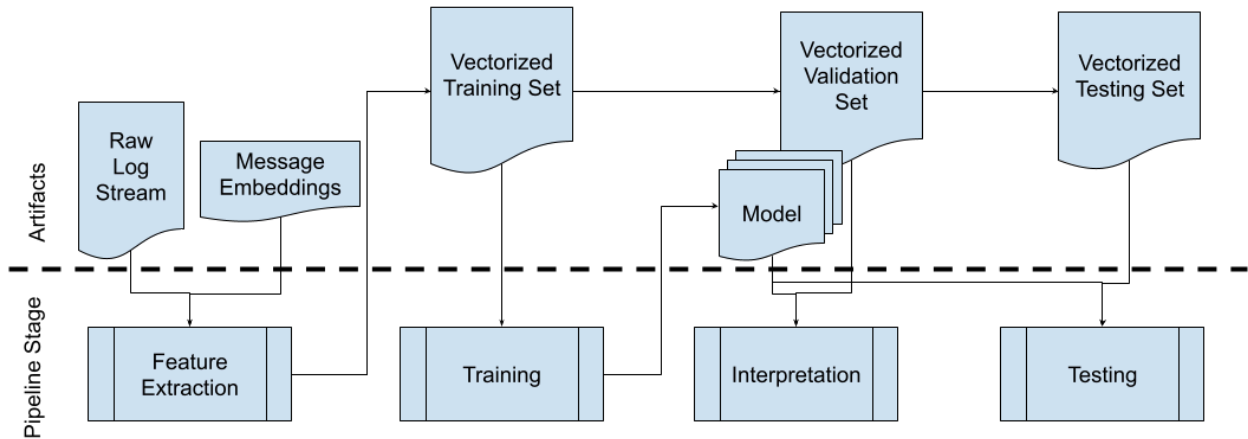


Figure 1: High-level Pipeline Architecture with intermediate artifacts listed.

Logs in the Thunderbird2 dataset come in as single-line messages. Each line can be represented as the tuple: (kind, proc_name, pid, message). Of these, we extracted (kind, message). Kind is a string enum, described in Table 1, and message is a variable length string. The message field was matched against a set of log templates drawn from the Loghub Thunderbird2 frequency analysis [Zhu 2023] as a starting point. Each template consists of a regular expression with $\langle * \rangle$ wildcards and a unique event identifier. The regex is applied to the message field after structured prefix fields are stripped using multiple extraction strategies to handle varying log line formats. We augmented the original 1,241 templates with 57 additional expressions targeting high-frequency unmatched messages. Of particular note, the three dominant anomaly state categories (“N_MAIL”, “R_MTT”, and “R_VAPI”) representing over 98% of all anomalous log lines, had 100% unmatched rates under the original template set. Our

augmentation brought all three to 0% unmatched, improving overall template match rate from 70.49% to 91.00%. Each matched log line is assigned a template ID (0–1,297), which serves as the state observation for the Markov chain model. We refer to these as “log states.” Unmatched lines are assigned template ID 0. Our final log mappings are provided in our submission as a renamed json file, tbird2_vocab_v2.txt.

Normal	“N-error”		“R-error”		
“_”	“N_AUTH”	“N_PBS_BAIL”	“R_CHK_DSK”	“R_EXT_INODE2”	“R_RIP”
	“N_CALL_TR”	“N_PBS_BFD1”	“R_ECC”	“R_GPF”	“R_SCSI0”
	“N_CPU”	“N_PBS_BFD2”	“R_EXT_FS”	“R_MPT”	“R_SCSI1”
	“N_LUS_LBUG”	“N_PBS_CON2”	“R_EXT_FS_ABRT1”	“R_MTT”	“R_SEG”
	“N_MAIL”	“N_PBS_EPI”	“R_EXT_FS_ABRT2”	“R_NMI”	“R_SERR”
	“N_NFS”	“N_PBS_SIS”	“R_EXT_FS_IO”	“R_PAG”	“R_VAPI”
	“N_OOM”		“R_EXT_INODE1”	“R_PAN”	

Table 1: enumeration of the “kind” in each log line.

Lemma 1: the directly observed transition rate of a finite sequence $\{s\}$ is an unbiased estimator

Proof: In the idealized environment with infinite execution time, an ordinary single-order Markov model with transition matrix $A \in \mathfrak{R}^{m \times m}$ will eventually enumerate all possible sequences. Consider a finite subset $\{\hat{e}\}$, and calculate from it transition matrix $E \in \mathfrak{R}^{m \times m}$ as the directly observed transition rate in $\{\hat{e}\}$. Note that

$$\sum_{i,j} A_{ij} = \sum_{i,j} E_{ij} = 1$$

Thus, we do not have m^2 degrees of freedom to optimize, but only $m(m - 1)$. Without loss of generality, let,

$$E_{i1} = 1 - \sum_{i=1, j=2} E_{ij}$$

Given the number of transitions n_{ij} in $\{\hat{e}\}$, we can now evaluate the log-likelihood function,

$$l(p) = \log L(p) = \sum_{i,j} n_{ij} \log E_{ij}$$

$$l(p) = \sum_{i=1} n_{i1} \log E_{i1} + \sum_{i=1, j=2} n_{ij} \log E_{ij}$$

Note:

$$\frac{\delta}{\delta E_{ij}} (\log E_{i1}) = -\frac{1}{E_{ij}}$$

Taking the partial at each particular transition,

$$\frac{\delta l}{\delta E_{ij}} = \frac{n_{ij}}{E_{ij}} - \frac{n_{i1}}{E_{i1}}$$

Setting to zero:

$$\begin{aligned} \frac{n_{ij}}{E_{ij}} &= \frac{n_{i1}}{E_{i1}} \\ \frac{n_{ij}}{n_{i1}} &= \frac{E_{ij}}{E_{i1}} \end{aligned}$$

Since the selection of $j = 1$ is arbitrary, it extends that

$$E_{ij} \propto \frac{n_{ij}}{\sum_j n_{ij}}$$

We train a model based on the result of Lemma 1. The resulting model consists of three sparse datasets:

1. π : the vector of initial states, of shape DIM
2. $S0_1$: A probability matrix representing all combinations of $P(s_0, s_1)$, of shape DIM x DIM
3. $S0_1_2$: A probability tensor representing all combinations of $P(s_0, s_1, s_2)$, of shape DIM x DIM x DIM

At interpretation time, we load all three datasets above into memory and generate the following:

- S_1C0 : probability matrix for $P(s_1|s_0)$ derived from π and $S0_1$, of shape DIM x DIM
- S_2C01 : probability matrix for $P(s_2|s_0, s_1)$ derived from $S0_1$ and $S0_1_2$, of shape DIM² x DIM

By loading π and generating each of S_1C0 , and S_2C01 , we can interpret the Markov Chain interpretation for up to three states. In a direct evaluation mode, the three-tuple (s_0, s_1, s_2) can be used with $S0_1_2$ to retrieve the likelihood that that sequence occurred in the training data. In the predictive mode, given a state pair (s_0, s_1) we predict the incidences of all states s_2 , emitted as a vector of new priors. In the naïve interpretation case, we can select the predicted state as the maximum value in the vector.

We evaluate the model in two modes. In the anomaly detection mode, each 300-line batch in the validation and test sets is scored by computing the mean log-likelihood of all overlapping 3-line windows within the batch under the trained transition model. Batches whose mean log-likelihood falls below a threshold are classified as anomalous. The threshold was selected on the validation set by maximizing F1 score, then applied once to the test set to produce final reported metrics. In the predictive mode, prediction accuracy is evaluated against the ground truth observed template across all overlapping 3-line windows in each batch. Epsilon smoothing of 0.001 is applied at inference time to assign small but non-zero probability to template transitions not observed during training, preventing undefined log-likelihood for unseen 3-grams.

Results

Based on the mappings described above, the 210,726,716-line corpus was parsed and vectorized, encoding each log line as a pair of integers: a state flag (0–33) representing the anomaly kind, and a template ID (0–1,297) representing the matched log message template. Of the 210,726,716 lines processed, 91.00% were successfully matched to a template; the remaining 9.00% were assigned template ID 0. All major anomaly state categories achieved 0% unmatched rate following template augmentation, with the residual unmatched lines consisting entirely of normal-state boot sequence, authentication, and system administration messages carrying no fault signal. The format of these messages was highly varied and did not lend itself to efficient regex pattern matching. As these lines carry no fault signal, we accepted a 9.8% unmatched rate within the normal class and trained on the successfully matched lines.

The encoded lines were divided into non-overlapping chunks of 300 sequential lines. Each chunk was labeled with one of four strata based on the state flags of its constituent lines: normal (all state flag 0), N-error only (contains N_ anomaly lines, no R_), R-error only (contains R_ anomaly lines, no N_), or mixed (contains both N_ and R_ anomaly lines). These strata are summarized in Table 3.

The chunks were partitioned 70/10/20 between training, validation, and testing sets. Initial analysis of a purely sequential partition revealed severe imbalance — the validation set contained only 0.13% anomalous chunks due to the temporal clustering of fault events in the corpus. To address this, the corpus was divided into 10 equal chronological segments, and within each segment chunks were assigned proportionally to each partition by stratum. This stratified approach produced anomaly rates of 8.30%, 10.29%, and 7.25% across training, validation, and testing respectively, as reported in Table 2.

Partition	Line Count	Normal Lines	Anomalous Lines	Anom%
Total	210,726,616	193,259,157	17,467,459	8.29%
Training	147,504,000	135,258,512	12,245,488	8.31%
Validation	21,071,000	18,903,527	2,167,473	10.3%
Testing	42,151,616	39,097,118	3,054,498	7.25%

Table 2: Comparative distributions of normal and anomalous lines within each partition.

Partition	Batch Count	Normal Batches	Mixed Batches	Anomalous Batches
Total	702,423	450,063	32,790	219,570
Training	491,680	315,041	22,948	153,691
Validation	70,237	45,003	3,280	21,954
Testing	140,506	90,019	6,562	43,925

Table 3: Comparative distributions of normal and anomalous lines within each partition.

Partition	Total Distribution	Normal Line Distrib	Anom. Line Distrib	Mixed Batch Distrib
Total	100%	99.98%*	100%	100%
Training	70.0%	70.0%	70.1%	70.0%
Validation	10.0%	9.78%	12.4%	10.0%
Testing	20.0%	20.2%	17.5%	20.0%

Table 4: Comparative distributions of total line, total anomalous line, and mixed batches between partitions.

*Totals may not add up to 100% due to rounding errors.

We evaluated the model in two phases: hyperparameter selection on the validation set, followed by a single evaluation pass on the held-out test set.

On the validation set of 70,237 batches (25,234 anomalous, 45,003 normal), we first established a majority class baseline: a classifier predicting normal for every batch achieves 64.07% accuracy with F1 of 0.0, providing no anomaly detection capability. We then swept two hyperparameters: the epsilon smoothing floor for unseen template transitions (values from $1/\text{DIM} \approx 0.00077$ to 0.1) and the aggregation method (plain mean, minimum, and trimmed mean at 5–30% trim). Epsilon values from 0.00077 to 0.05 produced negligible variation in F1 (0.529–0.536), confirming the false positive rate is structural rather than attributable to smoothing. At epsilon=0.1 the model collapsed entirely, predicting all batches as normal. The minimum aggregation method produced comparable F1 to mean but lower precision, while trimmed mean improved F1 marginally from 0.5361 to 0.5397 at 30% trim, which we deemed insufficient to justify additional complexity. We therefore selected plain mean log-likelihood with epsilon=0.001 and a F1-maximizing threshold of -3.790 as the locked configuration for testing. Across all configurations, PR-AUC ranged from 0.238 to 0.242, and ROC-AUC from 0.236 to 0.505. We report PR-AUC as the primary threshold-independent metric given the class imbalance in our dataset: with 35.93% of validation batches anomalous, PR-AUC more faithfully reflects classifier performance across the precision-recall tradeoff than ROC-AUC, which is sensitive to the large number of true negatives.

Validation results revealed the model's key strength and limitation. Among anomalous batches, detection rates were high across all three strata: 98.34% for N-error, 98.46% for R-error, and 100% for mixed batches, while only 5.07% of normal batches were correctly cleared. Overall binary precision was 0.3681, comparable to a naive always-anomalous baseline of 0.3593, indicating the binary threshold provides limited lift over the base rate. Score distributions of normal and anomalous batches overlap substantially, with a mean log-likelihood gap of 0.618 log units against standard deviations exceeding 1.0, making clean separation at any threshold infeasible. These findings motivated the hyperparameter sweep described above but confirmed the false positive rate is structural rather than addressable through threshold or aggregation method tuning.

On the test set of 140,506 batches (50,487 anomalous, 90,019 normal), the model achieved precision of 0.3583, recall of 0.9672, F1 of 0.5228, and PR-AUC of 0.2434, consistent with validation and indicating no overfitting to the validation threshold. For comparison, a naive always-anomalous classifier achieves F1 of 0.5283 on this dataset, confirming that binary thresholding provides limited lift over the base rate. Detection rates by batch type tell a more informative story: the model correctly detected 92.16% of N-error batches, 99.43% of R-error batches, and 99.68% of mixed batches, while correctly clearing only 2.83% of normal batches. The score distribution confirms why binary thresholding is ineffective: normal batches had mean window log-likelihood of -6.284 (std=1.082) versus -5.666 (std=1.440) for anomalous batches, a gap of 0.618 log units against standard deviations exceeding 1.0.

	Normal	N-error	R-error	Mixed
Total Batches (test)	90,019	19,034	24,891	6,562
Validation Detection Rate	5.07%	98.34%	98.46%	100.00%
Test Detection Rate	2.83%	92.16%	99.43%	99.68%
Next-state acc.	65.64%	72.47%	60.86%	60.79%

Table 5: Heatmap of detection rate and predictive quality by batch type of our final model.

In the predictive mode, the model correctly predicted the next template ID for 65.64% of overlapping 3-line windows overall (26,810,495 normal windows at 65.87% accuracy, 15,037,706 anomalous windows at 65.23%), far exceeding the random baseline of $1/1299 \approx 0.08\%$. Our test approach did not generate a prediction if the prior line pair was not observed in the training data, however, so this accuracy result is inflated by only including previously observed s0, s1 sequences. Per-stratum prediction accuracy revealed an unexpected pattern: N-error batches had higher prediction accuracy (72.47%) than normal batches (65.87%), while R-error and mixed batches were lower (60.86% and 60.79%). This is consistent with the repetitive nature of N_MAIL fault sequences, which once initiated produce highly predictable repeated template transitions. The small gap between normal and anomalous prediction accuracy (0.64 percentage points) indicates that predictability alone is insufficient to discriminate between normal and anomalous sequences — the absolute log-likelihood of transitions, rather than their predictability, is the more discriminating signal.

Discussion

Without significant optimization effort, we are able to generate a fully trained model taking only 22MB on disk in under four hours. Similarly, interpretation is very efficient, running around 5Kops/sec. All work was performed on consumer-grade hardware and without GPU acceleration. While not as flexible as an LLM-based approach, it was much less taxing on team resources to create.

By the definitions used for our training algorithm, resulting models will predict that unseen log transitions will appear at a rate of approximately α/count . As α tends towards 0, the resulting model predicts that unseen log transitions will never occur. This behavior results in events that are low-but-not-zero probability but are not seen in the training sequence. It occurs to the authors that the logical conclusion of such behavior is that we presently cannot collect information about the events that would be most interesting to any real-world operators of this system. That is, this model will not predict novel events given a log sequence, only what log is most likely to appear next.

It seems logical to the authors to extend our approach in several ways. Added model complexity could improve certain aspects of recall and detection. Additional complexity could take many forms, such as increasing the look-back depth/order of the Markov Chain, adding a hidden layer to the state model, and moving to a multi-model approach utilizing comparative likelihoods. Alternatively, different training methodologies may improve the operational guarantees of our system. We discuss each in brief below.

Increased look-back depth may allow for detection of longer-order code pathways, particularly in complex stateful systems. That is, log states that only occur after three or more prior log states will only be captured in relation to the latest two states. While the second-order markov model may be sufficient for small systems, larger systems with bigger state spaces would likely benefit from a higher-order treatment.

While we had initially considered a Hidden markov model for our work, we quickly realized that our problem formulation did not match the requirements for a hidden model. Particularly, our assumption that each log emission is a *direct* measurement of the system state rather than a probabilistic emission from some underlying state proved incompatible. While we could have added a hidden layer, it was unclear for the Thunderbird 2 dataset what that second layer would represent. In principle, a hidden layer *could* encapsulate information about the underlying state of the system, which we had taken as a probabilistic input. However, while logs themselves are well categorized, we do not have a corresponding dataset categorizing the underlying states. We hypothesize that this approach could work if augmented by the additions discussed below.

Taking a multi-model approach where the resulting output combines the log likelihoods from each model may improve prediction. As discussed in our Results, most of the resulting discriminatory signal comes from the log-likelihood for a given transition with certain classes of simple errors most likely to create strong predictions of next states. Given these asymmetries exist, it seems feasible that making several “stratified” models trained for each modality would provide strong predictive behaviors in smaller models at the cost of more complicated training and interpretation workflows.

Similarly, it’s likely worth separating different software components into independent models. While inter-system interaction grows increasingly common in our interconnected world, our assumptions account for that behavior under our “probabilistic inputs” assumption. Since modern Operating Systems are designed with multiprocessing as a core tenant [Englander 2009], we can at a first order reasonably describe different processes on one or more systems as operating as fully independent systems. Under such a model, we could have many much smaller Markov models each responsible for a single software component. While there may be concern for identifying the class of ordering errors between systems, we would consider those out-of-scope for initial work. Ontologically, the authors express concern over the wisdom of treating different instances of a single distributed system independently.

Moving to indirect approaches, basic tooling could enable significantly deeper integration with the underlying software. In this work, we assume that the log states observed from the training sequence are the only states possible. Any novel log states that appear at analysis time are mapped to our “null state” and are treated as equivalent to all other unseen states. This behavior does not mesh with the author’s understanding of how any real-world system operates. Notably, this behavior is inconsistent with our underlying assumption that code pathways on average tend to follow similar flows and thus emit similar logs. A tool based on either simple per-line regex of code artifacts or a more complex pipeline leveraging PL analysis tools such as [Strageloop’18/Tree-sitter] could extract the full set of possible log emissions.

Lastly, online model training and updating could enable our system to adapt to changes in the log flow. Live computer systems are rarely statically configured, with US Federal Guidelines such as [NIST-SP-800-53] calling for computer operators to perform regular software updates. New updates in general can include new code and new logging, resulting in a need to retrain a static point-in-time model like the one we have here after upgrades. Taken blindly, retraining results in loss of insights from previous software versions that may still hold value and increases load on system operators. By allowing a system to “learn online” its transition matrix, the system could retain that historical information while beginning to adapt to the new versions published.

Conclusion

This work steps us towards a robust set of solutions for performing log-based anomaly prediction. While overall precision and recall show space for improvement, our predictive analysis shows much stronger statistics, suggesting that we have successfully encoded some information about the Thunderbird 2 system's behavior into a very lightweight model. The model's lightness makes it a prime candidate for experimental adoption under other parameters and designs. Ultimately, our goal would be to look towards improving system performance and availability by providing live estimates of likely system transitions to its operators, allowing for live corrective action where necessary.

References

[CFDR/HPC4] <<https://www.usenix.org/cfdr-data#hpc4>>. Retrieved April 9, 2026.

[CFDR/HPC4/Tbird2]
<<http://0b4af6cdc2f0c5998459-c0245c5c937c5dedcca3f1764ecc9b2f.r43.cf2.rackcdn.com/hpc4/tbird2.gz>>.
Retrieved April 9, 2026.

[Englander 2009] *The architecture of Computer Hardware and Systems Software*. (2009) Irv Englander, published by Wiley.

[Github/LogHub-2.0] <<https://github.com/logpai/loghub>>. Retrieved April 10, 2026.

[NIST-SP-800-53] *NIST Special Publication 800-53 revision 5: Security and Privacy Controls for Information Systems and Organizations* (September 2020). US Department of Commerce Joint Task Force. DOI: 10.6028/NIST.SP.800-53r5. Retrieved from
<<https://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.800-53r5.pdf>> on April 10, 2026.

[Strageloop'18/Tree-sitter] *Tree-Sitter - A new parsing system for programming tools*, StrangeLoop (2018). Retrieved from
<<https://www.thestrangeloop.com/2018/tree-sitter---a-new-parsing-system-for-programming-tools.html>> on April 10, 2026.

[Zhu 2023] *Loghub: A Large Collection of System Log Datasets for AI-driven Log Analytics*, IEEE 34th Symposium on Software Reliability Engineering (2023). DOI: 10.1109/ISSRE59848.2023.00071; arXiv:2008.06448 [cs.SE]. Retrieved from <<https://ieeexplore.ieee.org/abstract/document/10301257>> on April 10, 2026.

[Zhang 2025] *Regular Expression Indexing for Log Analysis. Extended Version*, 11 October 2025 arXiv:2510.10348v1 [cs.DB]. Retrieved from <<https://arxiv.org/html/2510.10348v1>> on April 10, 2026.

[Liu 2020] *Valid Probabilistic Anomaly Detection Models for System Logs*, Wireless Communications and Mobile Computing, 16 November 2020. Retrieved from <<https://onlinelibrary.wiley.com/doi/10.1155/2020/8827185>> on April 10, 2026.

Team Roles

All members of our project team worked collaboratively, although each member had primary responsibility for different aspects of the project. MF was primarily responsible for data engineering and model validation/testing. YB performed core training work and generated initial versions of the model. TS led overall design of the project and project and presentation writing, and designed and implemented the model interpretation code.